UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

INFORME DE PRÁCTICAS

Automatización de Pruebas BDD con Cucumber y Python en Sistemas de Gestión Bibliotecaria

---

Asignatura: Ingeniería de Requisitos

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Período Académico: PPA 2025-2026

Estudiante:

Loor Medranda Marlon Teilor

Quevedo – Ecuador
2026

Índice

1. Investigación Bibliográfica **2**
1.1. Instalación en entornos .NET (C#) 2
1.2. Escritura de escenarios 2
1.3. Prueba y validación 2

2. Desarrollo de la Práctica **3**
2.1. Instalación del entorno 3
2.2. Configuración de Visual Studio Code 4
2.3. Escritura de escenarios en Gherkin 4
2.4. Implementación de los pasos en Python 4
2.5. Ejecución del escenario 5

3. Conclusiones **7**

Referencias **8**

Anexos y Evidencias Visuales del IDE **9**

SEMANA 14: Acoplamiento de Cucumber (Lenguaje Gherkin) con la plataforma de C#

1. Investigación Bibliográfica

Cucumber es una herramienta de automatización de pruebas funcionales basada en la metodología de Desarrollo Guiado por Comportamiento (BDD, por sus siglas en inglés). Su propósito es reducir la brecha entre los requisitos del negocio y su verificación técnica, permitiendo que los criterios de aceptación sean escritos en un formato legible tanto para el equipo técnico como para los interesados del proyecto [1].

El lenguaje que Cucumber emplea para este fin se denomina Gherkin. Se trata de un Lenguaje Específico de Dominio (DSL) que describe el comportamiento del sistema mediante palabras clave como *Feature, Scenario, Given, When* y *Then*. Mantener estas palabras en inglés es la práctica establecida en la industria, ya que los analizadores de Cucumber las reconocen en ese idioma de forma nativa, evitando errores de sintaxis en entornos de integración continua [2].

1.1. Instalación en entornos .NET (C#)

Para integrar BDD en proyectos C# del ecosistema Microsoft se utiliza **SpecFlow**, la implementación de Cucumber para la plataforma .NET [3]. Cabe mencionar que a partir de enero de 2025 el proyecto continúa bajo el nombre **Reqnroll** [4], manteniendo plena compatibilidad con la misma sintaxis Gherkin y los mismos frameworks de prueba. Los pasos de instalación son los siguientes:

1. Crear un proyecto de pruebas en Visual Studio compatible con NUnit, MSTest o xUnit.
2. Instalar el paquete `SpecFlow` (o `Reqnroll`) desde el gestor de dependencias NuGet.
3. Instalar el paquete integrador correspondiente al framework elegido (por ejemplo, `SpecFlow.NUnit`).
4. Habilitar la extensión de Visual Studio para obtener soporte visual y autocompletado en los archivos `.feature`.

1.2. Escritura de escenarios

Los requerimientos se transcriben en archivos con extensión `.feature` siguiendo la sintaxis Gherkin [1]. En C#, cada paso descrito en el escenario se enlaza a un método de clase mediante atributos en los llamados *Step Definitions*. De esta forma, cada oración en lenguaje natural se traduce en una aserción ejecutable dentro del código.

1.3. Prueba y validación

Una vez escritos los *Step Definitions*, las pruebas se ejecutan desde el *Test Explorer* de Visual Studio. Esta integración muestra en tiempo real qué escenarios pasan y cuáles fallan, lo que facilita detectar desviaciones entre el comportamiento esperado y el implementado. Además, puede incorporarse a pipelines de Integración Continua (CI/CD) para ejecutarse de forma automática con cada cambio en el repositorio [5].

SEMANA 15: Demostrar la automatización de pruebas en Cucumber + Python

2. Desarrollo de la Práctica

En esta semana se automatizó el Caso de Uso **CU-01: Registrar Préstamo Interno**, correspondiente al proyecto "Sistema Integral de Gestión, Control y Seguimiento de Lectura Interna en la Biblioteca Universitaria". Se utilizó Python como lenguaje de implementación junto con el framework **Behave** [6], que es el equivalente de Cucumber para dicho lenguaje.

2.1. Instalación del entorno

Se creó un entorno virtual con el módulo nativo `venv` de Python para aislar las dependencias del proyecto. Dado que el sistema operativo utilizado es Windows, fue necesario ajustar las políticas de ejecución de PowerShell antes de activar el entorno.

```
# 1. Creacion del entorno virtual
python -m venv venv

# 2. Desbloqueo de permisos de ejecucion (requerido en Windows)
Set-ExecutionPolicy Unrestricted -Scope CurrentUser

# 3. Activacion del entorno virtual
.\venv\Scripts\Activate.ps1

# 4. Instalacion del framework de BDD
pip install behave
```

Listing 1: Comandos ejecutados para preparar el entorno en Windows

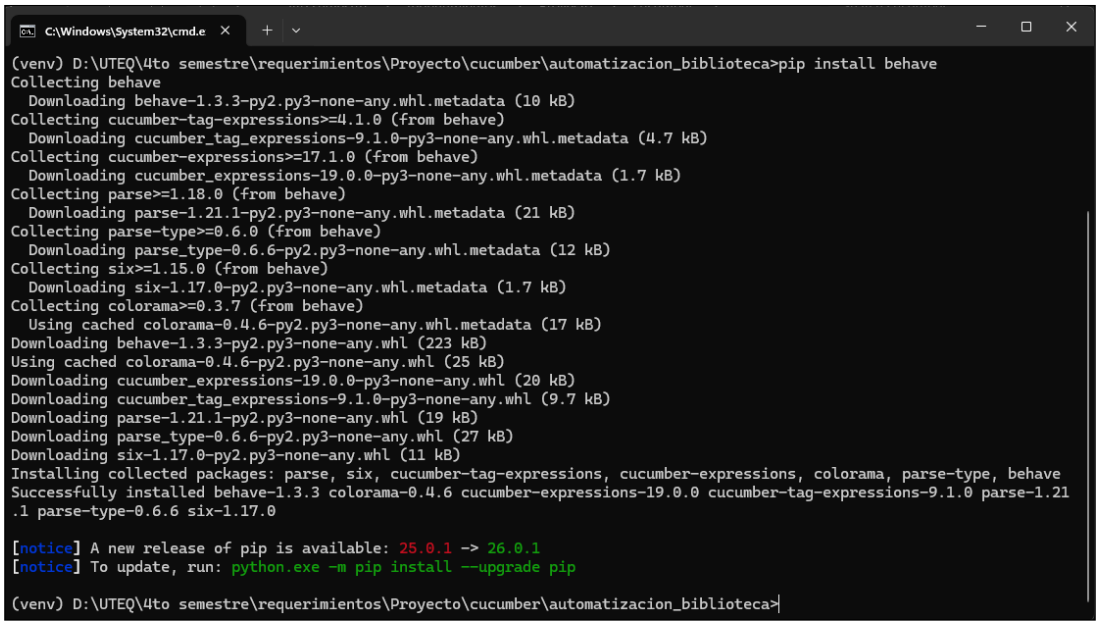

Figura 1: Instalación exitosa del framework Behave en el entorno virtual (terminal de Windows).

2.2. Configuración de Visual Studio Code

Una vez activo el entorno virtual, se configuró Visual Studio Code como IDE principal. Se instaló la extensión *Cucumber (Gherkin) Full Support* (ver Figura 4), que habilita el resaltado de sintaxis, autocompletado de palabras clave y detección de errores en los archivos `.feature`. Esta extensión reduce los errores tipográficos en las sentencias Gherkin y permite identificar de forma visual los pasos sin definición.

Adicionalmente, se definió la estructura de directorios del proyecto separando las especificaciones de negocio de su implementación técnica:

```
automatizacion_biblioteca/
|-- features/
|   |-- steps/
|   |   `-- prestamo_steps.py
|   `-- prestamo_interno.feature
`-- venv/
```

Listing 2: Estructura de directorios del proyecto de pruebas

Esta organización sigue la convención que Behave impone por defecto: busca los archivos `.feature` en el directorio `features/` y los *steps* en el subdirectorio `features/steps/`.

2.3. Escritura de escenarios en Gherkin

Se redactó el archivo `prestamo_interno.feature` respetando la sintaxis estándar de Gherkin. Las palabras clave se mantienen en inglés para garantizar la compatibilidad con el analizador de Behave.

```
Feature: Registrar Préstamo del Material
  Para formalizar la entrega de material bibliográfico
  Como Bibliotecario
  Quiero identificar al estudiante con su credencial y registrar el pré
     stamo

  Scenario: Registro exitoso de préstamo a un estudiante habilitado
    Given que el estudiante presenta su Credencial Digital Institucional
    And el estudiante no tiene sanciones activas
    When el bibliotecario valida la credencial
    And escanea el código del material bibliográfico
    Then el sistema muestra el mensaje "Habilitado para Préstamo"
    And el sistema vincula el material al estudiante para iniciar el pré
       stamo
```

Listing 3: Archivo prestamo_interno.feature

2.4. Implementación de los pasos en Python

Cada sentencia del escenario fue vinculada a una función Python en el archivo `prestamo_steps.py`. El decorador de cada función contiene el texto exacto del paso correspondiente en el `.feature`; Behave los empareja en tiempo de ejecución. Las funciones simulan las reglas de negocio: verificación de credencial y comprobación de sanciones activas.

```
from behave import given, when, then

@given('que el estudiante presenta su Credencial Digital Institucional')
def step_presentar_credencial(context):
```

```python
    # Simulamos que el estudiante entrego su credencial
    context.credencial_presentada = True

@given('el estudiante no tiene sanciones activas')
def step_sin_sanciones(context):
    # Simulamos la consulta a la base de datos (0 sanciones)
    context.sanciones_activas = False

@when('el bibliotecario valida la credencial')
def step_validar_credencial(context):
    assert context.credencial_presentada is True, "No hay credencial"
    if not context.sanciones_activas:
        context.estado_validacion = "Habilitado para Préstamo"
    else:
        context.estado_validacion = "Usuario Bloqueado"

@when('escanea el código del material bibliográfico')
def step_escanear_material(context):
    # Simulamos el uso del lector de codigo de barras
    context.material_escaneado = True

@then('el sistema muestra el mensaje "{mensaje}"')
def step_mostrar_mensaje(context, mensaje):
    # Verificamos que el mensaje del sistema coincida con el esperado
    assert context.estado_validacion == mensaje, \
        f"Error: Se mostro '{context.estado_validacion}'"

@then('el sistema vincula el material al estudiante para iniciar el pré
    stamo')
def step_vincular_prestamo(context):
    # Confirmamos que todo se cumplio para registrar el prestamo
    assert context.material_escaneado is True, "Falta escanear el libro"
    context.prestamo_registrado = True
    assert context.prestamo_registrado is True
```

Listing 4: Implementación de los pasos en Python (prestamo_steps.py)

2.5. Ejecución del escenario

La prueba se lanzó desde la terminal integrada de VS Code con el comando `behave`. Behave localizó el archivo `.feature`, identificó cada paso y ejecutó la función Python correspondiente en el orden definido por el escenario.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    powershell

(venv) PS D:\UTEQ\4to semestre\requerimientos\Proyecto\cucumber\automatizacion_biblioteca> behave
USING RUNNER: behave.runner:Runner
(venv) PS D:\UTEQ\4to semestre\requerimientos\Proyecto\cucumber\automatizacion_biblioteca> behave
(venv) PS D:\UTEQ\4to semestre\requerimientos\Proyecto\cucumber\automatizacion_biblioteca> behave
USING RUNNER: behave.runner:Runner
Feature: Registrar Préstamo del Material # features/prestamo_interno.feature:1
  Para formalizar la entrega de material bibliográfico
  Como Bibliotecario
  Quiero identificar al estudiante con su credencial y registrar el préstamo
    And el sistema vincula el material al estudiante para iniciar el préstamo # features/steps/prestamo
_steps.py:31 0.000s
    And el sistema vincula el material al estudiante para iniciar el préstamo # features/steps/prestamo
_steps.py:31
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
6 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
(venv) PS D:\UTEQ\4to semestre\requerimientos\Proyecto\cucumber\automatizacion_biblioteca>
```

Figura 2: Salida de la terminal tras ejecutar `behave`: todos los pasos aprobados.

La consola reportó *1 feature passed, 1 scenario passed, 6 steps passed*, confirmando que el flujo principal del caso de uso CU-01 fue validado sin errores. El resultado muestra que el escenario de registro de préstamo funciona según las reglas de negocio definidas: un estudiante con credencial válida y sin sanciones activas queda habilitado para recibir material bibliográfico.

3. Conclusiones

A partir de las actividades desarrolladas en las semanas 14 y 15 se obtienen las siguientes conclusiones:

- BDD con Gherkin permite que los criterios de aceptación de un caso de uso sean, al mismo tiempo, documentación legible y prueba ejecutable. Esto reduce la ambigüedad entre lo que el cliente solicita y lo que el equipo implementa.
- En el ecosistema .NET, SpecFlow cumple el mismo rol que Behave en Python: actúa como puente entre los archivos `.feature` escritos en Gherkin y el código de prueba del lenguaje anfitrión. La elección de uno u otro depende exclusivamente del lenguaje del proyecto.
- La configuración del entorno virtual en Windows requirió un paso adicional (ajuste de políticas de ejecución de scripts) que no es necesario en Linux o macOS. Este tipo de diferencias entre sistemas operativos deben considerarse al documentar los pasos de instalación para equipos de desarrollo heterogéneos.
- El escenario implementado para CU-01 valida únicamente el flujo principal. Para una cobertura completa sería necesario agregar escenarios alternativos, como el intento de préstamo por parte de un estudiante con sanciones activas o con credencial vencida.

Referencias

[1] Cucumber Open Source Project. "Gherkin Reference," visitado 19 de feb. de 2026. dirección: https://cucumber.io/docs/gherkin/reference/.

[2] Guru99. "Lenguaje Gherkin: formato, sintaxis y prueba de Gherkin en Cucumber," visitado 19 de feb. de 2026. dirección: https://www.guru99.com/es/gherkin-test-cucumber.html.

[3] Microsoft Tech Community. "What is BDD? How to use SpecFlow in Visual Studio 2022," visitado 19 de feb. de 2026. dirección: https://techcommunity.microsoft.com/blog/testingspotblog/what-is-bdd-how-to-use-specflow-in-visual-studio-2022---specflow-series---part-1/3255140.

[4] Reqnroll Contributors. "Reqnroll Documentation: BDD Automation for .NET." Reqnroll es el sucesor oficial de SpecFlow tras su retiro el 31 de diciembre de 2024, visitado 19 de feb. de 2026. dirección: https://docs.reqnroll.net/latest/.

[5] Profile Software Services. "Qué es Gherkin: cómo usarlo y cuáles son sus elementos," visitado 19 de feb. de 2026. dirección: https://profile.es/blog/que-es-gherkin/.

[6] Behave Contributors. "Behave Documentation: Behavior-driven development, Python style," visitado 19 de feb. de 2026. dirección: https://behave.readthedocs.io/en/latest/.

Anexos y Evidencias Visuales del IDE

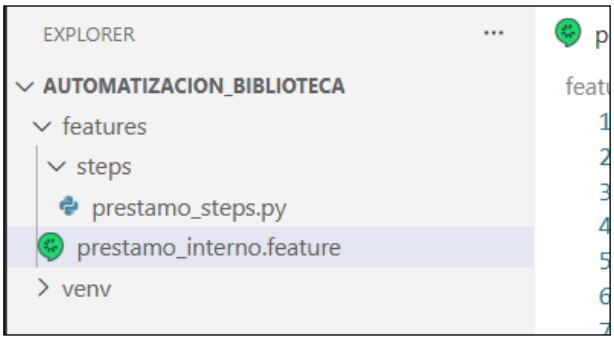

Figura 3: Estructura de directorios generada en VS Code para el proyecto de pruebas.

Figura 4: Instalación de la extensión Cucumber (Gherkin) Full Support en VS Code.

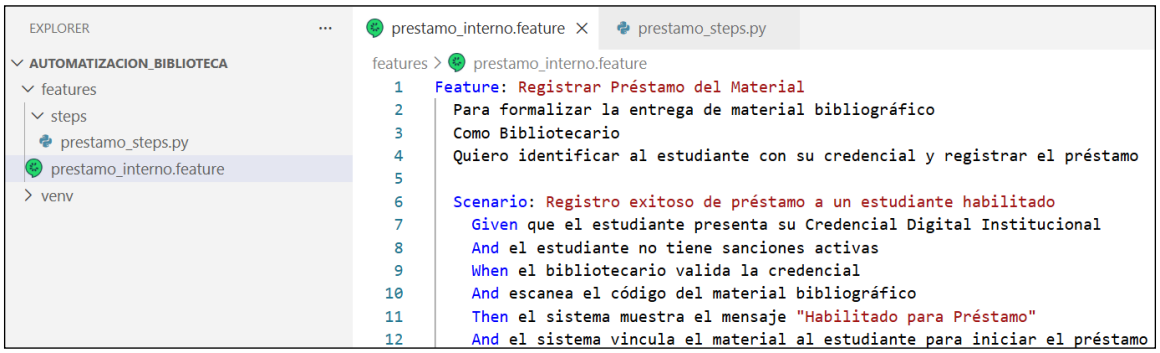

Figura 5: Visualización del archivo `.feature` con soporte de sintaxis activo.